DUET: Dual Manifold Alignment for Recoverable Vision-Language-Action Policies

Anonymous Authors

***Abstract*— Vision-language-action (VLA) models map vision and language to robot actions. However, after post-training on limited robot demonstrations, VLA policies may learn to imitate in-distribution behavior without acquiring the ability to recognize out-of-distribution (OOD) states or recover from them. Existing recovery systems often rely on external OOD predictors, verifiers, or world models for detection, and train recovery behavior from manually collected intervention and OOD-recovery trajectories. These pipelines incur substantial data and engineering cost, while their recovery modules remain loosely coupled to the action policy. We introduce DUET (Dual-manifold Unified Execution and Transfer), which jointly learns a state-aware relative tangent field and a stateless absolute waypoint manifold, and uses their endogenous disagreement together with state-distribution distance to detect OOD and shift control toward the stable absolute anchor for recovery. We evaluate DUET in LIBERO simulation and real-world robot manipulation across diverse designed OOD events and scenarios. DUET preserves nominal success (TODO% vs. TODO% for the relative-only baseline) while achieving TODO% OOD recovery success, outperforming the strongest recovery baseline by TODO percentage points.**

I. INTRODUCTION

Vision-language-action (VLA) policies adapt pretrained vision-language models to map visual observations and language instructions directly to robot actions [1], [2], [3]. While this formulation enables broad semantic generalization, real robot execution is closed loop and frequently encounters conditions not represented in the task-specific demonstrations used for post-training. We call an execution state out-of-distribution (OOD) when its visual observation, robot–object configuration, or induced trajectory departs from the support of this post-training distribution. OOD can arise from transient camera occlusion, unexpected object displacement, external displacement of the robot arm, or execution deviations accumulated during contact and long-horizon control. In these situations, a VLA may continue to produce locally plausible actions even though the evolving trajectory is no longer supported by its demonstrations. Enabling the policy to recognize OOD and then either actively recover or remain stable can therefore substantially improve task completion and deployment reliability.

Existing approaches commonly formulate OOD detection as an auxiliary prediction problem, introducing an external monitor, critic, verifier, or predictive model and training it with curated nominal and OOD rollouts [4]. Collecting and annotating OOD data is costly because the space of possible visual, object, and robot-state shifts is combinatorial. Recovery introduces a second data bottleneck: interactive imitation learning typically requires a human operator to monitor policy rollouts and provide takeover or corrective trajectories from OOD states [5], [6], [7]. Beyond their data and supervision cost, these pipelines learn detection and recovery as separate modules, so the signal used to declare OOD is not necessarily aligned with the action required to leave it. We instead seek a single policy that uses information endogenous to its own action prediction process to jointly identify OOD and recover, without collecting additional OOD or intervention data.

DUET realizes this idea by learning the same demonstrated behavior in two complementary action spaces, as illustrated in Fig. 1. Relative controls, such as delta operational-space-control commands, form a smooth local tangent field for precise nominal execution. Absolute waypoints instead provide coordinates on a global demonstrated state manifold. Near the demonstration support, the relative tangent action agrees with the projection induced by the absolute waypoint; as execution moves OOD, their geometric inconsistency grows. DUET treats this endogenous disagreement as an intrinsic OOD signal and uses the global waypoint manifold as a stable recovery anchor. Crucially, the absolute branch is stateless with respect to the current robot pose, preventing it from collapsing into the shortcut "absolute waypoint = current state + delta" and preserving its role as an observation-conditioned global anchor.

At inference time, DUET converts each absolute waypoint into a correction from the live pose and compares it with the relative tangent action. Tangent–projection disagreement and distance from the training state bounds jointly determine a soft fusion weight. The controller follows the relative branch under nominal conditions, shifts toward manifold projection under OOD, and returns smoothly to relative control after re-entering the demonstrated support. Detection and recovery are therefore two consequences of the same cross-space inconsistency rather than two separately trained capabilities.

This draft makes the following contributions:

- A dual-manifold VLA formulation that interprets relative delta actions as local tangent vectors and stateless absolute waypoints as global state-manifold anchors.
- A training recipe that combines residual vector-quantized (RVQ) condition anchoring, classifier-free guidance (CFG), and visual OOD augmentation with gradient truncation on the tangent-field branch.
- A closed-loop inference rule that uses tangent-projection disagreement and state-distribution distance to adaptively fuse relative control with absolute recov-

This paper is an ICRA 2026 draft. Author names and affiliations are omitted for anonymous review.

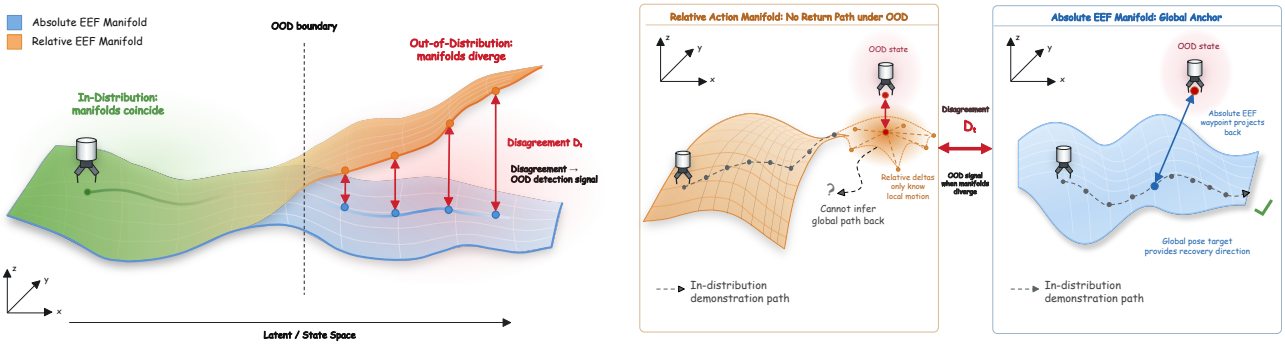

Fig. 1. Dual-manifold OOD detection and recovery. The relative and absolute end-effector manifolds coincide in-distribution but diverge under OOD, exposing an endogenous disagreement score D_t. Relative actions provide precise local motion but no global return path, whereas absolute waypoints project execution toward a stable demonstrated-manifold anchor.

ery.

- An implementation in the StarVLA framework and an evaluation in LIBERO simulation and real-world manipulation across visual, object, and robot-state OOD events.

II. RELATED WORK

A. OOD Detection for Robot Policies

Runtime OOD detection estimates whether a policy remains within the support of its training distribution. DiffDAgger uses diffusion uncertainty to request expert intervention [8], while Sentinel monitors action inconsistency and task progress [9]. FAIL-Detect treats detection as sequential OOD testing and calibrates policy-derived signals using only successful executions [4]; FIPER similarly combines representation novelty with action uncertainty without failure data [10].

VLA-specific methods use internal features or external foundation models. SAFE learns a detector from successful and failed rollouts [11], whereas AHA trains a VLM to detect and diagnose failures [12]. Such methods require additional supervision or a separate detector whose output is not a recovery action. DUET instead detects OOD through disagreement between two action spaces learned within the policy, directly measuring inconsistency between local control and a global manifold anchor.

B. Recovery from OOD Execution

Recovery methods commonly expand the training distribution with expert corrections. DAgger queries experts on policy-induced states [5]; intervention-based methods collect human takeovers [13], [6]; and IntervenGen transforms limited interventions into corrective trajectories [7]. These approaches reduce compounding errors but still require additional recovery actions.

Foundation-model systems instead use semantic supervisors. RACER learns from language-annotated recovery trajectories [14], while AHA and VLM controllers diagnose OOD and guide downstream correction [12], [15]. Their detection and recovery components remain separately trained or prompted. DUET couples both capabilities without extra recovery data: absolute states from successful demonstrations provide recovery anchors, while cross-space disagreement determines when and how strongly to move toward them.

III. METHOD

A. Problem Setup

At time t, the robot observes multi-view images o_t, a language instruction l, and a current proprioceptive state s_t. The policy predicts an action chunk of horizon $K = 8$. We use two target representations:

$$\Delta a_{t:t+K-1} \in \mathbb{R}^{K\times 7}, \tag{1}$$

$$W_{t+1:t+K} \in \mathbb{R}^{K\times 7}, \tag{2}$$

where Δa denotes relative OSC commands and W denotes future absolute end-effector waypoints with dimensions $[x, y, z, r, p, y, g]$. The gripper channel in W is treated as an absolute gripper state during training, but the deployed gripper command is taken from the relative branch.

B. Two Action Manifolds

Let $\mathcal{M}$ denote the demonstrated end-effector state manifold induced by successful task trajectories, and let $T\mathcal{M}$ denote its local tangent bundle. A relative action policy learns a vector field

$$f_{\text{rel}}(o_t, l, s_t) \in T_{s_t}\mathcal{M}, \tag{3}$$

which is most reliable when s_t lies near the support of demonstrations. This representation is locally expressive: it specifies how to move from the current chart, but it does not by itself encode where the global trajectory manifold should be.

An absolute waypoint policy instead predicts a future coordinate on the state manifold,

$$f_{\text{abs}}(o_t, l) = \hat{W}_{t+1:t+K} \in \mathcal{M}^K. \tag{4}$$

The branch is intentionally stateless with respect to s_t, so its prediction is a visual-language-conditioned manifold anchor rather than a current-state-relative displacement. When the robot is perturbed off manifold, the vector from the live pose to this absolute waypoint becomes an empirical projection direction back toward $\mathcal{M}$.

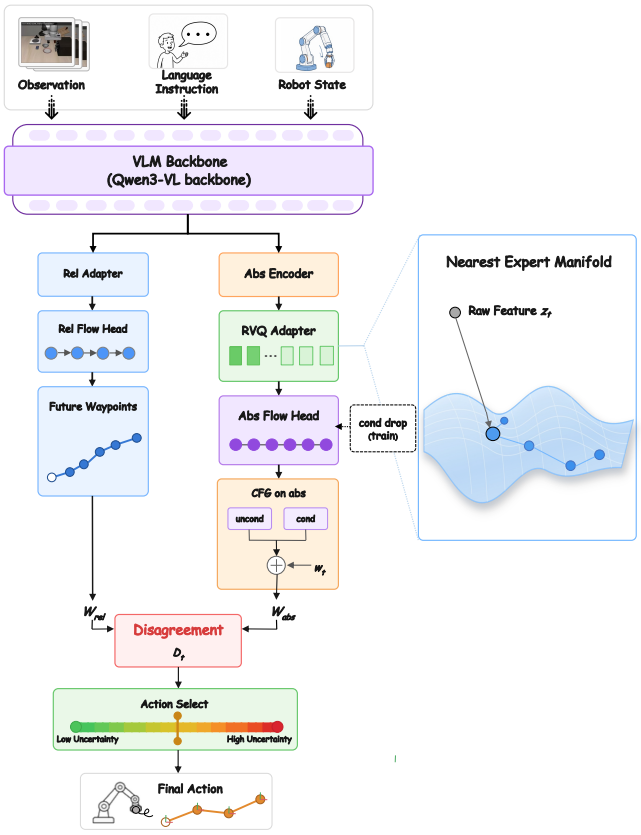

Fig. 2. DUET architecture. A shared VLM feeds a state-aware relative flow branch and a stateless absolute flow branch anchored by RVQ and refined with CFG. Their cross-space disagreement D_t drives adaptive action selection between local tangent following and global waypoint recovery.

DUET aligns these two geometries online. The relative branch supplies a tangent-following command; the absolute branch supplies a manifold-projection command. Their disagreement estimates whether the local tangent field still agrees with the global demonstrated manifold. Low disagreement indicates that local execution is consistent with the task manifold, while high disagreement signals that a projection-dominated recovery action should take over.

C. Dual-Branch Architecture

DUET uses a shared Qwen-VL backbone to encode images and instruction tokens, as shown in Fig. 2. The relative branch attends to the full VLM hidden sequence and receives the normalized current state through a numerical state encoder. The absolute branch receives only RVQ-anchored condition tokens produced from state-free VLM features.

The relative branch is intended to be the primary controller under nominal conditions. It learns a smooth local vector field on the action tangent space using a 16-layer DiT-B flow-matching action head [16], [17] with action dimension 7 and horizon 8. The absolute branch predicts a state-manifold projection target using a smaller 8-layer DiT-B flow-matching head with state dimension zero. Its output is clamped in normalized space to $[-1, 1]$, matching the min-max normalized absolute-state labels.

D. Stateless Absolute Branch

The central design invariant is:

> *The absolute branch must not condition on the current robot state s_t.*

DUET supports two state pathways. First, s_t enters the relative branch through a continuous state encoder. Second, when enabled, normalized s_t is formatted as a text segment and appended to the end of the VLM prompt. Because the VLM is causal, earlier token representations do not attend to this final state segment. The absolute branch then constructs a key-padding mask that excludes the state marker, every token after it, and padding positions from RVQ pooling. Thus the relative branch can exploit state-aware VLM features, while the absolute branch remains stateless.

E. RVQ Condition Anchoring

Let $H \in \mathbb{R}^{B\times L\times d_h}$ be the final VLM hidden sequence after masking out state-contaminated positions for the absolute branch. A learned-query pooling module maps H to $N = 16$ latent condition tokens:

$$Z_e = \mathrm{Pool}(H) \in \mathbb{R}^{B\times N\times d_c}. \tag{5}$$

DUET applies residual vector quantization [18], [19] with $M = 3$ quantizers:

$$R_0 = Z_e, \tag{6}$$

$$Q_m = \mathrm{VQ}_m(R_{m-1}), \tag{7}$$

$$R_m = R_{m-1} - Q_m, \tag{8}$$

$$Z_q = Z_e + \left(\sum_{m=1}^{M} Q_m - Z_e\right)_{\mathrm{sg}}, \tag{9}$$

where sg denotes stop-gradient. The commitment loss is

$$\mathcal{L}_{\mathrm{vq}} = \beta \left\| Z_e - \mathrm{sg}\left(\sum_{m=1}^{M} Q_m\right)\right\|_2^2. \tag{10}$$

The final residual norm $\|R_M\|$ is recorded as an auxiliary OOD score. Intuitively, the quantizer maps arbitrary visual-language conditions to nearby codebook combinations, giving the absolute branch a discrete atlas over demonstrated conditions. The waypoint head then predicts from an anchored chart rather than from an unconstrained continuous condition vector.

F. Classifier-Free Absolute Waypoint Head

During training, the absolute head applies classifier-free guidance training [20] by randomly replacing the quantized condition tokens with learnable null tokens with probability $p_{\mathrm{drop}} = 0.15$. At inference time, its flow velocity is computed as

$$v = v_{\mathrm{uncond}} + w\,(v_{\mathrm{cond}} - v_{\mathrm{uncond}}), \tag{11}$$

where w is the classifier-free guidance scale. DUET adapts w online using the previous OOD estimate, allowing the absolute branch to move toward a more conservative unconditional state-manifold prior when the robot is strongly OOD.

IV. TRAINING OBJECTIVE

Each LIBERO training sample provides the relative action chunk and a sequence of current plus future states. We use the dataset action as the relative label and future states $s_{t+1:t+K}$ as absolute waypoint labels after selecting dimensions $[0, 1, 2, 3, 4, 5, 7]$ and min-max normalizing them to $[-1, 1]$.

The total loss is

$$\begin{aligned}\mathcal{L} = \lambda_{\text{rel}}\mathcal{L}_{\text{rel}} + \lambda_{\text{abs}}\mathcal{L}_{\text{abs}} + \lambda_{\text{vq}}\mathcal{L}_{\text{vq}} \\ + \lambda_{\text{gate}}\mathcal{L}_{\text{gate}},\end{aligned} \tag{12}$$

where $\mathcal{L}_{\text{gate}}$ is optional. The default implementation uses equal weights for the relative, absolute, and VQ terms. Both action losses are flow-matching objectives.

A. OOD Augmentation with Relative Gradient Truncation

For each batch, DUET applies visual perturbations to a subset of samples with probability $p_{\text{aug}} = 0.3$. Perturbations include image noise, brightness/contrast/color jitter, and random crop-resize. The absolute branch and RVQ adapter train on both clean and augmented samples, so the manifold anchor remains stable under degraded observations. The relative tangent-field branch computes loss only on clean samples:

$$\mathcal{L}_{\text{rel}} = \frac{1}{|\mathcal{B}_{\text{clean}}|} \sum_{i \in \mathcal{B}_{\text{clean}}} \ell_{\text{FM}}(\hat{\Delta a}^{(i)}, \Delta a^{(i)}). \tag{13}$$

This preserves the relative branch's sensitivity to OOD observations. If both branches were trained to adapt to the same perturbations, their tangent-projection disagreement would become less informative at test time.

B. Optional Learned Gate

DUET includes an optional second-stage gate. A small MLP receives detached VLM pooled features concatenated with the current state. During training, states are perturbed with Gaussian noise and the gate is supervised with a binary label indicating whether the state was perturbed. At deployment, the gate score can be averaged with the heuristic OOD score.

V. CLOSED-LOOP MANIFOLD ALIGNMENT

The policy server returns both unnormalized action representations: a relative chunk $\Delta a_{t:t+K-1}$ and an absolute waypoint chunk $W_{t+1:t+K}$. At each environment step, the client compares the live pose p_t with the current absolute waypoint and converts the difference into an action-space projection command:

$$c_t = \text{clip}\left(\frac{W_t - p_t}{\sigma_a}, -1, 1\right), \tag{14}$$

where σ_a contains the position and rotation action scales. In implementation, rotation correction is computed using the geodesic relative rotation rather than an element-wise axis-angle difference.

TODO: Insert gate response plot.
Expected curves: injected perturbation interval, disagreement D_t, state distance S_t, fusion weight α_t, and CFG scale w_t over time.

Fig. 3. Planned closed-loop telemetry visualization for perturbation recovery experiments.

The tangent-projection disagreement is

$$D_t = \frac{\left\| c_t^{1:6} - \text{clip}(\Delta a_t^{1:6}, -1, 1) \right\|_2}{\sqrt{6}}. \tag{15}$$

This quantity is an empirical estimate of manifold inconsistency: it is small when the local tangent vector points in the same direction as the global projection command, and large when local control no longer agrees with the state-manifold anchor. The state-distribution distance measures normalized violation of the training state min-max bounds:

$$S_t = \text{mean}\left[\frac{\max(s_{\min} - p_t, 0)}{s_{\max} - s_{\min}} + \frac{\max(p_t - s_{\max}, 0)}{s_{\max} - s_{\min}}\right]. \tag{16}$$

The OOD score and adaptive fusion weight are

$$q_t = D_t + \lambda S_t, \tag{17}$$

$$\alpha_t = \sigma(k(q_t - \tau)). \tag{18}$$

The final action softly transitions between tangent following and manifold projection:

$$a_t^{1:6} = (1 - \alpha_t)\Delta a_t^{1:6} + \alpha_t c_t^{1:6}, \tag{19}$$

with the gripper command copied from the relative branch. When requesting a new action chunk, the client schedules the CFG scale as

$$w_t = w_{\max} - (w_{\max} - w_{\min})\alpha_{t-1}. \tag{20}$$

VI. EXPERIMENTS

A. Benchmark and Data

We evaluate on LIBERO [21], using the StarVLA LeRobot data pipeline. The dual-label data configuration uses current plus future state indices $0, \ldots, 8$, while action labels use the native delta OSC actions. The default draft configuration trains on the `libero_10_hybrid` mixture, with planned extensions to `libero_all_hybrid`. For few-shot experiments, the data loader limits demonstrations per task through `max_demos_per_task`.

B. Implementation Details

DUET is implemented as `QwenGR00THybrid` in StarVLA. The backbone is Qwen-VL with hidden dimension 2048. The relative branch uses a 16-layer DiT-B flow-matching head. The absolute branch uses an 8-layer DiT-B flow-matching head with 16 RVQ condition tokens of output dimension 1024. RVQ uses three residual quantizers with codebook size 512 and code dimension 256. Training uses AdamW and the schedule specified in the repository configuration.

TABLE I

LIBERO SUCCESS RATES. ALL QUANTITATIVE ENTRIES ARE TODO PLACEHOLDERS AND MUST BE FILLED AFTER RUNNING EVALUATION.

Method	Demos/task	Clean SR	Perturbed SR
Relative-only VLA	TODO	TODO	TODO
Absolute-only waypoint	TODO	TODO	TODO
Fixed fusion, $\alpha = 0.5$	TODO	TODO	TODO
DUET adaptive	TODO	TODO	TODO
DUET + learned gate	TODO	TODO	TODO

TABLE II

PLANNED ABLATIONS. TODO ENTRIES SHOULD BE FILLED WITH SUCCESS RATE, RECOVERY RATE, AND TELEMETRY STATISTICS.

Ablation	Clean SR	Perturbed SR
DUET full	TODO	TODO
No RVQ anchoring	TODO	TODO
No CFG adaptation	TODO	TODO
No relative gradient truncation	TODO	TODO
State-to-VLM disabled	TODO	TODO
Fixed $\alpha = 0$	TODO	TODO
Fixed $\alpha = 1$	TODO	TODO

C. Perturbation Recovery Protocol

To test recovery, we inject an action-space perturbation for a fixed interval during rollout. The evaluation script supports perturbation start step, duration, and magnitude. We record success, action traces, dual-branch disagreement, state-distance score, fusion weight, and CFG scale for each episode.

D. Main Results

Table I will report the main clean and perturbation recovery success rates. The expected analysis is that relative-only control should be competitive on clean rollouts but less robust under perturbation, while DUET should improve recovery by increasing the manifold-projection weight only when needed.

E. Ablations

The ablations in Table II isolate the contribution of condition anchoring, guidance adaptation, branch specialization, state prompt injection, and adaptive fusion. Additional metrics should include mean maximum displacement after perturbation, mean recovery time, and area under the α_t curve.

VII. LIMITATIONS

DUET assumes access to absolute state labels aligned with the relative action chunks. This is natural in LIBERO and many robot logs, but may require additional calibration or state estimation on real hardware. The closed-loop gate currently uses simple min-max state statistics as a coarse proxy for distance to the demonstrated state manifold, which may be insufficient for highly multi-modal workspaces. The absolute branch is deliberately stateless; while this supports recovery, it may reduce waypoint accuracy in tasks where the current robot state contains task-relevant information not recoverable from images. Finally, all quantitative claims in this draft remain pending until the planned LIBERO and perturbation experiments are completed.

VIII. CONCLUSION

We presented DUET, a dual-branch VLA policy that aligns a local relative-action tangent field with a stateless absolute-waypoint manifold. By enforcing state isolation in the absolute branch, anchoring its condition with RVQ, and using online tangent-projection disagreement to gate fusion and guidance, DUET turns two complementary action geometries into a closed-loop recovery mechanism. The implementation is integrated into StarVLA and is ready for LIBERO few-shot and perturbation recovery evaluation.

ACKNOWLEDGMENT

TODO: Add acknowledgments, funding, compute resources, and any required dataset or codebase acknowledgments after the author list is finalized.

REFERENCES

[1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, "RT-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, T. Lampe, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, C. Finn, and P. Liang, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[3] StarVLA Community, "StarVLA: A lego-like codebase for vision-language-action model developing," https://github.com/starVLA/starVLA, 2026.

[4] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina, "Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies," in *Proceedings of Robotics: Science and Systems*, 2025.

[5] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, 2011.

[6] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, "Robot learning on the job: Human-in-the-loop autonomy and learning during deployment," in *Proceedings of Robotics: Science and Systems*, 2023.

[7] R. Hoque, A. Mandlekar, C. R. Garrett, K. Goldberg, and D. Fox, "IntervenGen: Interventional data generation for robust and data-efficient robot imitation learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024, pp. 2840–2846.

[8] S.-W. Lee, X. Kang, and Y.-L. Kuo, "Diff-DAgger: Uncertainty estimation with diffusion policy for robotic manipulation," in *IEEE International Conference on Robotics and Automation*, 2025.

[9] C. Agia, R. Sinha, J. Yang, Z. Cao, R. Antonova, M. Pavone, and J. Bohg, "Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress," in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 689–723.

[10] R. Römer, A. Kobras, L. Worbis, and A. P. Schoellig, "Failure prediction at runtime for generative robot policies," in *Advances in Neural Information Processing Systems*, 2025.

[11] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti, "SAFE: Multitask failure detection for vision-language-action models," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[12] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, "AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation," in *International Conference on Learning Representations*, 2025.

[13] J. Spencer, S. Choudhury, M. Barnes, M. Schmittle, M. Chiang, P. Ramadge, and S. Srinivasa, "Learning from interventions: Human-robot interaction as both explicit and implicit feedback," in *Proceedings of Robotics: Science and Systems*, 2020.

[14] Y. Dai, J. Lee, N. Fazeli, and J. Chai, "RACER: Rich language-guided failure recovery policies for imitation learning," in *IEEE International Conference on Robotics and Automation*, 2025, pp. 15 657–15 664.

[15] H. Chen, Y. Yao, R. Liu, C. Liu, and J. Ichnowski, "Automating robot failure recovery using vision-language models with optimized prompts," *arXiv preprint arXiv:2409.03966*, 2024.

[16] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, 2024.

[17] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, J. Jakubczak, T. Jones, A. Khazatsky, S. Levine, A. Li-Bell, Y. Lu, S. Nair, K. Pertsch, Q. V. Xu, F. Xia, and T. Xiao, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[18] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems*, 2017.

[19] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2022.

[20] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *arXiv preprint arXiv:2207.12598*, 2022.

[21] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, 2023.